

Three Ways to Miss a Grasp: Comparing Rules, a Trained Network, and GPT-4o on the Same Robot Grasping Test

Anish Talla

Abstract—A robot has to decide where to put its fingers before it can pick anything up. I compared three ways of making that decision from a single color photo: hand-written rules, a neural network trained on labeled grasps, and GPT-4o, which has never been trained on grasping. All three were scored on the same 123 test images of 35 objects from the Cornell Grasping Dataset, with the same scoring code. The trained network was right on 84.0% of images, averaged over three training runs. The rules were right on 57.7%. GPT-4o was right on 12.4% of 615 calls. They also failed in different ways. The rules could not produce diagonal grasps. The network usually had the angle right and the position slightly off. GPT-4o put the center of its grasp outside the area covered by every labeled grasp on 53.4% of calls, compared with about 7% for the other two. At first I thought GPT-4o could see a good grasp and just could not write it as pixel coordinates. To check, I drew numbered candidate grasps on each image and asked it to pick one. It did no better than picking at random on three different sets of candidates, and it kept choosing a grasp that runs along the length of the object near one end. The weak point is the choice of grasp itself.

I. INTRODUCTION

Before a gripper closes, something has to pick a contact point, an angle, and how wide to open. There are at least three ways to get that answer. A programmer can write rules, for example that a mug is grasped by its handle. A neural network can learn the mapping from thousands of labeled pictures. Or a vision-language model (VLM) such as GPT-4o, trained on text and images from the web and never on a grasping dataset, can be asked in plain English where to grip.

The third option is tempting because it costs almost nothing to set up. Rules take a person's time, and a trained network needs labeled data and a computer to train on. A VLM needs an API key. That is a good reason to measure it before trusting it.

Most grasp detection papers compare one trained network against another [1], [2]. Systems that use a VLM usually hand the final geometry to some other component, so it is hard to tell what the VLM contributed by itself [3], [4]. I wanted a comparison where all three kinds of system see the same test images, give the same kind of answer, and are graded by the same code. I had two questions. Which approach works best? And when each one fails, how does it fail? The second question ended up being the more interesting one, and it led to a fourth experiment that I had not planned.

II. METHODS

A. Data and scoring. I used the Cornell Grasping Dataset, which has 885 photos of household objects on a

white platform [5]. Each photo comes with several hand-labeled grasp rectangles, since most objects can be picked up more than one way. A rectangle records the grasp center, its angle, how far the gripper opens, and how wide the fingertips are. I used only the color images and ignored the depth data.

A predicted rectangle counts as correct if its angle is within 30 degrees of a labeled rectangle and the two overlap enough, meaning an intersection over union (IoU) above 25% [6]. Matching any one labeled grasp on the image is enough. This is the standard Cornell test, so I did not have to invent my own answer key and then grade myself against it.

The test has to use objects the systems have never seen. Cornell does not say which photos show the same object, so I had to work that out. I separated each object from the platform and compared its shape and color from one photo to the next. When the comparison was unclear I looked at the photos myself. I set the thresholds so that any mistakes would merge two objects instead of splitting one, because a split object could end up in both training and test. This gave 234 objects in 883 photos (two photos were dropped). Whole objects were assigned at random to training (620 photos), validation (140), and test (123 photos of 35 objects).

B. The four systems. System A uses rules. A Faster R-CNN detector pretrained on COCO [7] finds the object, and a fixed table says where to grasp each category. COCO does not know most Cornell objects, so when the detector finds nothing, System A falls back on the outline of the object against the platform and closes across the narrow side of its bounding box. It can only output 0 or 90 degrees. Its first test score was 40.7%. When I looked at its predictions I saw that the detector sometimes boxed clutter in the background. I added a check that the box has to contain the object's center and recalibrated one constant on training data, and the score went up to 57.7%. I made that fix after seeing test results, so 57.7% is not a clean held-out number. I report both.

System B is a ResNet [8] pretrained on ImageNet, with new output layers for the grasp center, size, and angle. The angle is predicted as the cosine and sine of twice the angle, because a gripper turned 180 degrees is the same grasp [1]. When an image has several labeled grasps, the loss uses only the one closest to the prediction. Averaging them would aim the network at a point between two good grasps. I trained it in two rounds. Round 1 trained ResNet18 and ResNet34 once each and kept the checkpoint with the best validation score. That turned out to be unreliable, since validation accuracy jumped around by 7 to 12 points from one epoch to the next. Round 2 used the same data and model with a steadier recipe: a cosine learning rate schedule over a fixed 100 epochs, an averaged copy of the weights, three random seeds for each network, and no picking of checkpoints. At test time it also combined predictions from eight flipped and rotated copies of the image. I wrote down how the final model would be chosen before I opened the test set the second time.

System C is GPT-4o [9], called as openai/gpt-4o through the OpenRouter API in August 2026 at the default temperature. It received the 640 by 480 photo and one prompt asking for the pixel positions of the two fingertips and a fingertip width, returned as JSON. Those values define

the rectangle. I wrote the prompt using 30 training images and then froze it. Every test image was sent five separate times, 615 calls in total. If a reply could not be parsed I counted it as wrong and did not ask again. The full prompts are in the Appendix.

System D came later, after I saw how System C failed. It uses the same model and images, but GPT-4o no longer writes coordinates. I drew numbered candidate grasps on a zoomed-in copy of the photo and asked for the number of the best one, similar to Set-of-Mark prompting [3]. The candidates came only from the object's outline and never from the labels. I found the object's center and its long axis, placed candidates at the center and toward each end, and gave each position four closing directions 45 degrees apart. The numbers were reshuffled on every call. Since the candidates are fixed before the model sees them, I can calculate what random guessing would score (the floor) and how often at least one candidate is correct (the ceiling). I ran three versions: twelve candidates at three positions, four candidates at the center drawn at their real size, and the same four drawn at one fixed size with a sentence in the prompt saying the size means nothing.

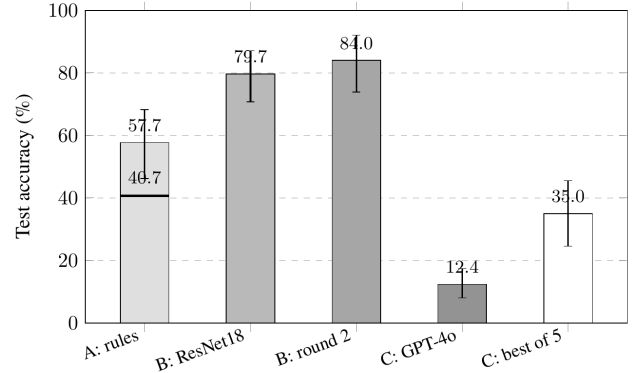
C. Data analysis. The 123 test photos are not independent, because they show 35 objects from several angles. All 95% confidence intervals here come from a bootstrap that resamples whole objects 20,000 times, and the five GPT-4o calls for a photo always stay together. I compared systems using differences computed on the same resamples and a permutation test that swaps labels object by object. I also sorted every wrong prediction by what it missed: the angle, the overlap, or both. Last, I checked whether the predicted center fell inside the area enclosed by all the labeled grasps on that photo, which is about the easiest spatial test a prediction could pass.

III. RESULTS AND DISCUSSION

Table I and Fig. 1 show the accuracies. In round 1, ResNet18 was correct on 98 of 123 photos (79.7%) and ResNet34 on 70.7%. I originally explained that gap by saying the smaller network suited a small dataset. Round 2 showed I was wrong. With three seeds each, ResNet34 averaged 84.0% (84.6, 83.7, 83.7) and ResNet18 averaged 82.6%, so the nine point gap had been luck. System A was correct on 71 photos. GPT-4o was correct on 76 of 615 calls. At least one of its five calls was correct on 43 photos (35.0%), but using that would take five calls and some way of knowing which call to believe. Round 2 of System B beat System A by 26.3 points, with an interval of 14.8 to 38.3 ($p = 0.0002$). System A beat a single GPT-4o call by 45.4 points (33.8 to 55.6, $p < 0.0001$).

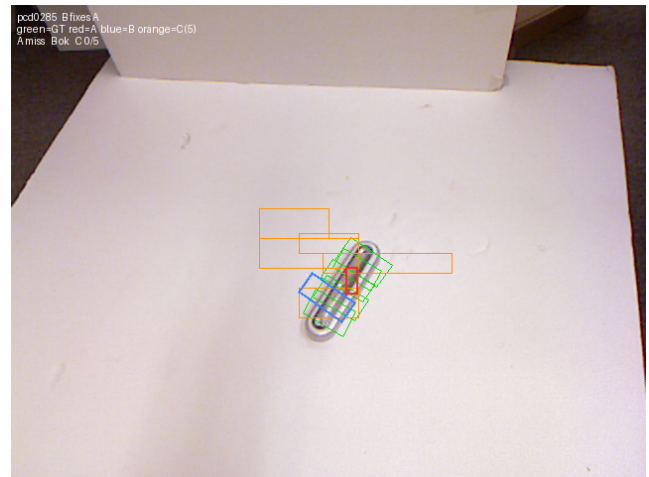
I. TEST ACCURACY WITH 95% CONFIDENCE INTERVALS FROM RESAMPLING WHOLE OBJECTS

System	Accuracy	95% interval
A: rules (40.7% before fix)	57.7%	46.2 to 68.2
B: ResNet18, round 1	79.7%	70.8 to 87.1
B: ResNet34, round 2	84.0%	73.9 to 92.1
C: GPT-4o, one call	12.4%	8.0 to 17.4
C: GPT-4o, best of five	35.0%	24.6 to 45.5



1. Test accuracy with 95% intervals from resampling whole objects. The line across the System A bar marks its 40.7% score before the fix. "B: round 2" is the ResNet34 average over three seeds. "C: best of 5" needs five calls and a way to know which one is right.

The kinds of mistakes differed more than I expected. System A had 52 misses, and 13 of them overlapped a labeled grasp well but had the wrong angle, which makes sense for something limited to two angles. System B had 25 misses in round 1. Only 4 were angle mistakes and 15 were placement mistakes. GPT-4o missed both the angle and the overlap on 365 of 615 calls (59.3%). Its grasp center was outside the labeled area on 53.4% of calls. For Systems A and B that number was 7.0% and 6.5% (Fig. 2 shows one example). To be fair, A and B start from the object's location or from training on this dataset, and GPT-4o can put its answer anywhere in the image. GPT-4o also disagreed with itself. Two calls on the same photo gave matching rectangles only 22.0% of the time.



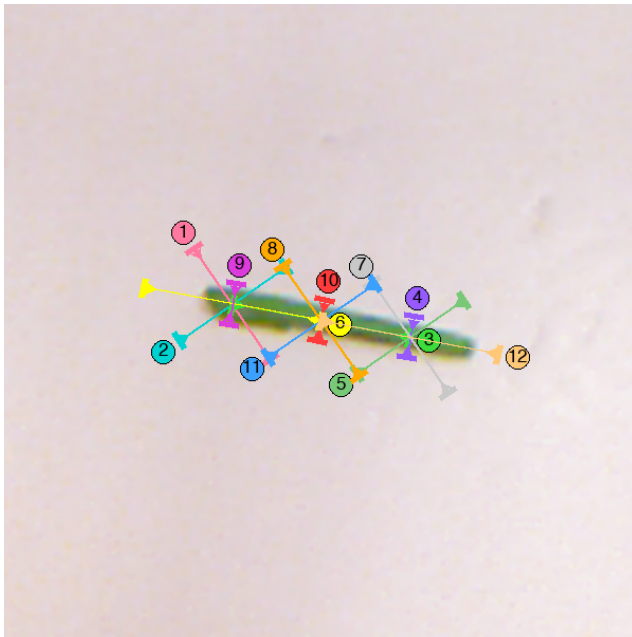
2. One test photo with every system's answer. Labeled grasps are green, System A is red, System B is blue, and System C's five calls are orange.

When I read GPT-4o's one sentence explanations, they often named a sensible part of the object while the coordinates in the same reply pointed somewhere else. My guess was that the model knew where to grasp and lost the location when it had to write numbers. System D was built to test that guess, and the guess did not hold up (Table II, Fig. 3). With twelve candidates, GPT-4o picked a correct one

on 18.7% of calls. Random guessing would get 20.1%, and a correct candidate was available on 79.7% of photos. The four candidate versions scored 26.5% and 22.3% against a floor of 24.6%. None of the three differences from random can be told apart from zero.

II. SYSTEM D ON THREE SETS OF CANDIDATES. LENGTHWISE IS THE SHARE OF CALLS THAT CHOSE A GRASP CLOSING ALONG THE OBJECT'S LENGTH

Menu	Correct	Floor	Ceiling	Lengthwise
12 marks	18.7%	20.1%	79.7%	70%
4 marks, real size	26.5%	24.6%	65.9%	41%
4 marks, same size	22.3%	24.6%	65.9%	45%



3. A test photo as System D saw it, with twelve numbered candidates. GPT-4o chose mark 6, the grasp along the bean near its end, on all five calls. Mark 10 crosses the bean at its center and is correct.

Its picks were not random, though. With twelve candidates, 61% of its choices were the same type: a grasp near one end whose fingers close along the length of the object. That type is correct 17% of the time. The candidate at the center that closes across the narrow direction is correct on 61% of photos, and GPT-4o chose it on 6% of calls. I wondered if it just liked bigger marks, since on thin objects the correct marks are the smallest ones. That is why the third version drew every mark the same size. It still chose the lengthwise direction 45% of the time when chance would be 25%. It said its confidence was high on 556 of 560 calls.

Here is my best explanation. In a photo taken from above, two fingertips on either side of the end of a pen look like they are pinching something narrow. In three dimensions one of those fingers would come down on top of the pen. The model seems to judge the grasp as a flat picture. Writing coordinates was never the real problem. GPT-4o

picks the wrong grasp to begin with, at least from a single overhead view.

One side result surprised me. The center candidate that closes across the narrow direction uses no learning and no detector, and it scored 55.3% (41.8 to 68.0). That is about the same as System A, and unlike System A it can handle objects lying diagonally.

This study has real limits. It uses one split of a small dataset and color images without depth, and no robot ever tried these grasps. Cornell's labels do not cover every workable grasp, so some predictions marked wrong might work in practice. System B got far more tuning than the single frozen prompts used for GPT-4o, and I opened the test set twice for System B. System A's 57.7% includes a fix made after looking at test results. These results describe GPT-4o with my prompts, not VLMs in general, and methods that pass the geometry to other software do much better [4]. The next thing I would try is giving the model depth information.

ACKNOWLEDGMENT

I used AI tools in this project and want to be specific about how. GPT-4o is the model being tested (Systems C and D). Separately, I used Anthropic's Claude Code (Claude models, July to September 2026) as a coding and writing assistant. It helped write and debug the Python code, made a first pass at deciding which photos showed the same object, which I then checked by hand as described in the Methods, and helped draft and edit the text of this paper. OpenAI Codex helped shorten an earlier version. I chose the question, designed the experiments, ran them, checked the numbers against the saved outputs, and am responsible for everything written here.

REFERENCES

1. J. Redmon and A. Angelova, "Real-time grasp detection using convolutional neural networks," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2015, pp. 1316-1322.
2. S. Kumra, S. Joshi, and F. Sahin, "Antipodal robotic grasping using generative residual convolutional neural network," in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2020, pp. 9626-9633.
3. J. Yang et al., "Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V," arXiv:2310.11441, 2023.
4. M. Kulshrestha et al., "VLAD-Grasp: Zero-shot grasp detection via vision-language models," arXiv:2511.05791, 2025.
5. Y. Jiang, S. Moseson, and A. Saxena, "Efficient grasping from RGBD images: Learning using a new rectangle representation," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), 2011, pp. 3304-3311.
6. I. Lenz, H. Lee, and A. Saxena, "Deep learning for detecting robotic grasps," Int. J. Robotics Research, vol. 34, no. 4-5, pp. 705-724, 2015.
7. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems, vol. 28, 2015.
8. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.
9. OpenAI, "GPT-4o system card," arXiv:2410.21276, 2024.

APPENDIX: PROMPTS GIVEN TO GPT-4O

Model openai/gpt-4o through OpenRouter, default temperature, no seed. In the System D prompt, {k} is the number of candidates drawn. The version with every mark the same size adds one sentence to the System D prompt, shown last.

System C, system message. You are a robotic grasping assistant. You are shown a photograph from a fixed camera mounted above a table, and you decide where a two-fingered parallel-jaw gripper should close on the object in the photograph. You reply with a single JSON object and no other text.

System C, user prompt. This is a 640 x 480 RGB photograph of one object resting on a table, taken from a fixed camera looking down at it. A two-fingered parallel-jaw gripper will descend vertically and close on the object. Decide where the two fingertips should make contact. Coordinates are in pixels of this 640 x 480 image. x runs from 0 at the left edge to 639 at the right edge. y runs from 0 at the top edge to 479 at the bottom edge. Report whole numbers. Reply with exactly one JSON object and nothing else: { "object": "<two or three words naming the object>", "reasoning": "<one sentence: which part of the object you are grasping, and why>", "finger_a": [x, y], "finger_b": [x, y], "jaw_width_px": <integer>, "force": "<low/medium/high>", "confidence": "<low/medium/high>" } Rules: - finger_a and finger_b are the two points where the fingertips touch the object. The gripper closes along the line between them, so they must be on OPPOSITE sides of the part you are grasping. Do not put both points at the object's centre, and do not put both on the same side. - The distance between finger_a and finger_b is how far the jaws open. It must be wider than the part being grasped and no more than 150 pixels. - jaw_width_px is how much of the object the flat face of one fingertip covers, measured perpendicular to the closing direction. Usually 15 to 40. - Grasp the object itself. Do not place a fingertip on the table, on the background, or on the object's shadow. - If more than one grasp would work, choose the one most likely to hold: across a narrow part of the object, close to its centre of mass.

System D, system message. You are a robotic grasping assistant. You are shown a photograph of one object on a table, taken from a camera above it, with several candidate grasps drawn on the object as numbered coloured lines. You choose one candidate. You reply with a single JSON object and no other text.

System D, user prompt. This photograph shows one object on a table, zoomed in. A two-fingered parallel-jaw gripper will descend vertically and close on the object. {k} candidate grasps are drawn on the image, each in its own colour with a numbered circle beside it. Each candidate shows the gripper's two fingertips as two short THICK bars. The thin line joining the two bars is the direction the fingertips move: the arrowheads on that line point inward, and the two fingertips slide toward each other along it until they squeeze whatever lies between them. So a candidate grips the part of the object that lies between its two thick bars. Choose the candidate most likely to pick the object up and hold it: usually one whose two bars sit on opposite sides of a narrow part of the object, near its centre of mass, so that the fingertips squeeze the object rather than closing on empty table. Reply with exactly one JSON object and nothing else: { "object": "<two or three words naming the object>", "reasoning": "<one sentence: which part of the object lies between that candidate's two bars, and why it is a good grip>", "choice": <the number of the chosen candidate, 1 to {k}>, "confidence": "<low/medium/high>" }

Sentence added for the same-size version. All candidates are drawn at the same length, which is NOT to scale: a mark shows only where the grip is centred and the direction the fingertips close along. So a candidate grips the part of the object that the line between its two bars crosses, at the marked centre.